

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ**

Кафедра Систем информатики

Направление подготовки 09.06.01 – Информатика и вычислительная техника

ОТЧЕТ

Обучающегося Якутиной Светланы Александровны группы № 22215 курса 4
(Ф.И.О. полностью)

Тема задания: Разработка драйвера для графовой базы данных Neo4j

Новосибирск 2025

Оглавление

Введение	3
1 Реализация модуля работы с Neo4j	4
Заключение	7
Список литературы	8

Введение

Целью данной работы является разработка драйвера на языке Python для работы с графовой базой данных Neo4j.

Для реализации цели были поставлены следующие задачи:

- разработка вспомогательных функций для генерации URI и преобразования свойств/меток;
- создание методов для работы с узлами (создание, чтение, обновление, удаление);
- реализация методов для работы с дугами (создание, удаление, выборка);
- обеспечение возможности выполнения произвольных Cypher-запросов.

1 Реализация модуля работы с Neo4j

Для взаимодействия с Neo4j был разработан класс `Neo4jRepository`. В его состав входят:

- Вспомогательные функции

- `generate_random_string(length: int = 12) -> str` — генерирует уникальный идентификатор (`uri`) для нового узла.
- `transform_labels(labels: List[str], separator: str = ":") -> str` — преобразует список меток узла (`labels`) в строку для Cypher-запроса.
- `transform_props(props: Dict[str, Any]) -> str` — преобразует словарь свойств узла в формат Cypher.

Эти функции используются внутри методов CRUD для формирования корректных запросов к базе.

- Методы сбора данных (`collect`)

- `collect_node(node) -> TNode` — возвращает словарь `TNode`.
- `collect_arc(rel) -> TArc` — возвращает словарь `TArc`.

Эти методы позволяют работать с результатами запросов в Python в удобном виде.

- CRUD-операции для узлов

- `create_node(self, props: Dict[str, Any], labels: Optional[List[str]] = None) -> TNode` — создаёт узел с заданными свойствами и метками.
- `get_all_nodes(self) -> List[TNode]` — возвращает список всех узлов.
- `get_nodes_by_labels(self, labels: List[str]) -> List[TNode]` — возвращает узлы с заданными метками.
- `get_node_by_uri(self, uri: str) -> Optional[TNode]` — возвращает конкретный узел по его уникальному URI.
- `update_node(self, uri: str, props: Dict[str, Any]) -> Optional[TNode]` — обновляет свойства узла по URI.
- `delete_node_by_uri(self, uri: str) -> bool` — удаляет узел по URI вместе со всеми его связями.

Методы для работы с узлами обеспечивают создание, чтение, обновление и удаление.

- CRUD-операции для дуг

- `create_arc(self, node1_uri: str, node2_uri: str, arc_type: str = "RELATED") -> Optional[TArc]` — создаёт дугу заданного типа между двумя узлами.
- `delete_arc_by_id(self, arc_id: str) -> bool` — удаляет дугу по её идентификатору (`element_id`).

Методы для работы с дугами обеспечивают создание и удаление связей между узлами.

- Произвольные запросы

- `run_custom_query(self, query: str, params: Dict[str, Any] = None) -> List[Dict[str, Any]]` — позволяет выполнить любой Cypher-запрос с параметрами и возвращает результат в виде списка словарей Python.

Это даёт возможность расширять функционал без изменения класса.

Фрагмент кода класса `Neo4jRepository` приведён в листинге 1.

```
def create_node(self, props: Dict[str, Any], labels: Optional[List[str]] = None) -> TNode:
    labels = labels or []
    if "uri" not in props:
        props["uri"] = self.generate_random_string()
    label_part = f":{self.transform_labels(labels)}" if labels else ""
    props_part = self.transform_props(props)
    query = f"CREATE (n{label_part} {props_part}) RETURN n"
    with self.driver.session() as session:
        rec = session.run(query).single()
        return self.collect_node(rec["n"])
```

Листинг 1 – пример реализации метода `create_node`

При запуске метода создаётся новый узел с указанными свойствами и метками, после чего возвращается словарь с его данными.

Заключение

В результате выполнения работы были выполнены следующие задачи:

- реализованы основные CRUD-операции для узлов и дуг;
- обеспечена поддержка произвольных Cypher-запросов;
- реализованы вспомогательные функции для преобразования данных в корректный формат Cypher.

Список литературы

1. neo4j. Resource: <https://neo4j.com/>